

ResiGest

Guía de arquitectura, estructura y hoja de ruta
del **backend** en Java + Spring Boot

Documento de planificación técnica

Sin código: estructura, clases, métodos, interfaces, herencia y diagramas UML

Universidad Industrial de Santander — Ingeniería de Software I

Grupo 5 — Proyecto ResiGest — 2026

Contenido

1. Cómo usar este documento y por dónde empezar
2. Decisiones de arquitectura
3. Estructura de paquetes y carpetas
4. Convenciones de nombres y responsabilidades por tipo de clase
5. Modelo de dominio por módulo (clases, atributos y métodos)
 - 5.1 Común (base, seguridad, errores)
 - 5.2 Usuarios e Inmueble
 - 5.3 Financiero
 - 5.4 Reservas
 - 5.5 Mantenimiento
 - 5.6 Portería
 - 5.7 Comunicaciones y notificaciones
6. Dónde aplicar herencia y dónde aplicar interfaces
7. Contratos REST (endpoints por módulo)
8. Seguridad: autenticación JWT y matriz de permisos
9. Manejo de errores, validación y auditoría
10. Base de datos, migraciones y mapeo de entidades a tablas
11. Estrategia de pruebas
12. Hoja de ruta: 9 sprints semana a semana
13. Tu primera funcionalidad de punta a punta (paso a paso)
14. Errores frecuentes y cómo evitarlos
15. Anexo A: checklist de avance
16. Anexo B: glosario técnico mínimo

1. Cómo usar este documento y por dónde empezar

Este documento no es un tutorial de Spring Boot: es el **plano de construcción** del backend de ResiGest. Está pensado para que, aunque todavía estés aprendiendo el framework, siempre sepas tres cosas: qué clase te toca crear ahora, qué responsabilidad tiene, y cómo se conecta con las demás.

La idea central: no construyas el sistema por capas, constrúyelo por rebanadas

El error más común en un primer proyecto grande es intentar “hacer todas las entidades”, luego “todos los repositorios”, luego “todos los servicios”. Al final tienes cientos de archivos y nada funciona. La alternativa es la **rebanada vertical**: eliges una funcionalidad pequeña (por ejemplo “registrar un apartamento”) y la llevas de punta a punta —entidad, repositorio, servicio, controlador, prueba— hasta que responde en Postman. Cuando termines la primera rebanada, ya conoces el patrón y las siguientes se vuelven mecánicas.

Orden de lectura sugerido

1. Secciones 2, 3 y 4: te dan el mapa mental y las convenciones. Léelas completas antes de escribir nada.
2. Sección 13: la primera rebanada vertical, paso a paso. Hazla hoy mismo.
3. Sección 5: consúltala como referencia cada vez que empieces un módulo nuevo, no de corrido.
4. Sección 12: la hoja de ruta, para saber en qué semana debe estar cada cosa.

Sobre el alcance. Los documentos de la Entrega 1A/1B definen 8 requerimientos funcionales y 12 casos de uso. Este plan cubre los 8 RF. Sin embargo, si el semestre se aprieta, el orden de sacrificio recomendado es: primero el chat en tiempo real (WebSocket), luego las notificaciones push, luego el repositorio documental. El núcleo que **no** se negocia es: autenticación por roles (RF01), cartera y pagos (RF02) y reservas (RF03-RF04), porque son los que demuestran reglas de negocio reales.

Prerrequisitos de tu entorno

Herramienta	Versión sugerida y para qué
JDK	Java 21 (LTS). Spring Boot 3.x lo exige a partir de Java 17; con 21 tienes records y pattern matching maduros.
Spring Boot	3.3.x o superior. Genera el esqueleto en start.spring.io .
Maven	Incluido con el wrapper (<code>mvnw</code>). No instales Maven aparte.
PostgreSQL	16, corriendo en Docker para no ensuciar tu máquina.
IDE	IntelliJ IDEA Community. El autocompletado de anotaciones te va a ahorrar horas.
Postman / Insomnia	Para probar endpoints antes de que exista el frontend de tus compañeros.
DBBeaver o pgAdmin	Para ver qué está creando Hibernate realmente en la base.

Dependencias que debes marcar en Spring Initializr

```

Spring Web          -> controladores REST
Spring Data JPA      -> repositorios y mapeo objeto-relacional
Spring Security      -> autenticación y autorización por rol
Validation           -> @NotNull, @Email, @Positive... en los DTO
PostgreSQL Driver    -> conexión a la base de datos
Lombok              -> menos código repetitivo (getters, builders)
Spring Boot DevTools -> recarga automática mientras desarrollas
Flyway Migration     -> versionado del esquema de base de datos

Agregar manualmente al pom.xml más adelante:
jjwt (io.jsonwebtoken) -> generar y validar tokens JWT
springdoc-openapi-starter -> documentación Swagger automática
mapstruct            -> conversión entidad <-> DTO
itext o openpdf      -> generación de comprobantes PDF
testcontainers        -> pruebas de integración con PostgreSQL real
  
```

2. Decisiones de arquitectura

2.1 Monolito modular, no microservicios

Para un equipo de tres personas y un semestre, los microservicios son una trampa: multiplican el trabajo de infraestructura sin aportar nada evaluable. La decisión es un **monolito modular**: un solo proyecto, un solo despliegue, una sola base de datos, pero organizado internamente en módulos con fronteras claras. Si mañana quisieran separar el módulo financiero, la frontera ya está trazada.

2.2 Arquitectura por capas dentro de cada módulo

Cada módulo repite la misma estructura de cuatro capas. La regla que no se rompe nunca: **una capa solo depende de la capa que tiene debajo**. Un repositorio jamás llama a un servicio; un controlador jamás toca un repositorio directamente.

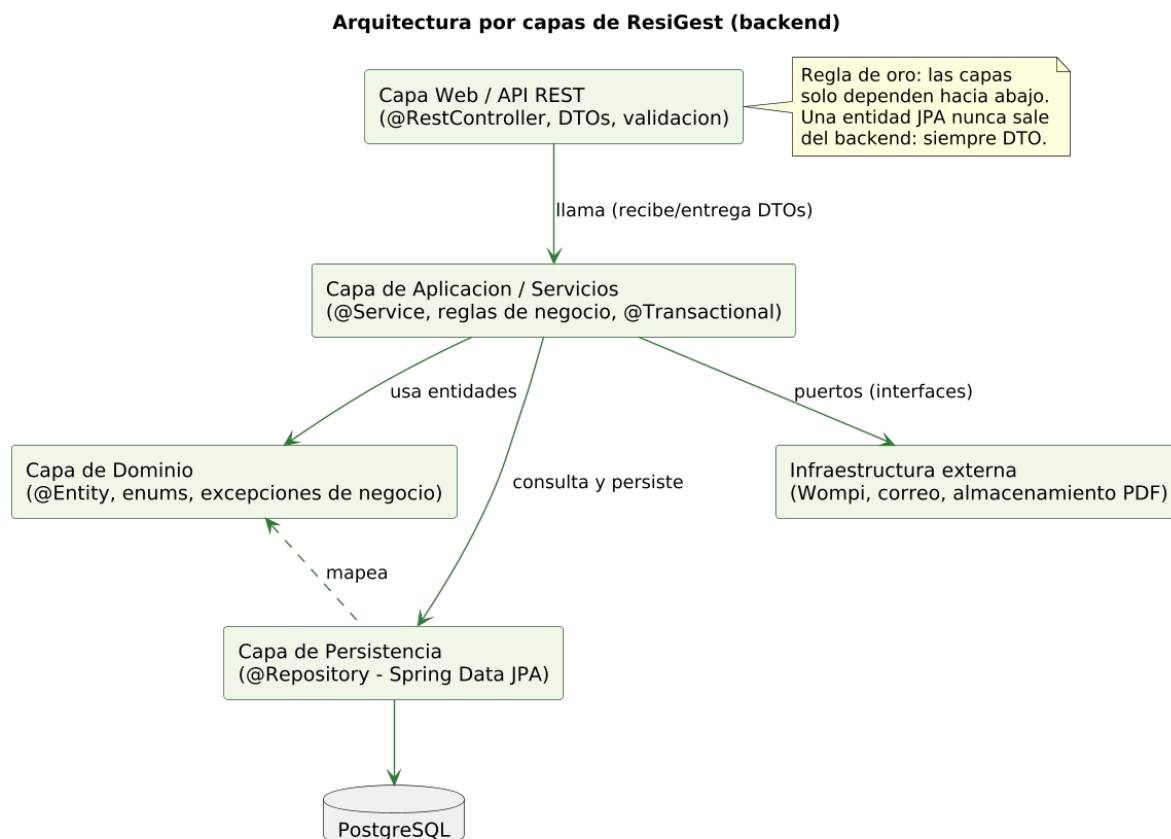


Figura 1. Capas del backend y dirección permitida de las dependencias.

Capa	Qué contiene	Qué le está prohibido
Web	@RestController, DTOs de entrada/salida, validación de formato.	Contener reglas de negocio o cálculos. Devolver entidades JPA.
Servicio	@Service, reglas de negocio, orquestación, @Transactional.	Saber que existe HTTP (nada de HttpServletRequest ni códigos de estado).
Dominio	@Entity, enums, invariantes propias del objeto.	Depender de repositorios o servicios.
Persistencia	Interfaces JpaRepository, consultas derivadas y @Query.	Tomar decisiones de negocio.

2.3 Módulos y sus dependencias

El módulo de comunicaciones es el punto delicado: los cuatro módulos operativos necesitan notificar, pero si cada uno llama directamente al servicio de notificaciones quedan acoplados. La solución es **eventos de dominio**: el módulo financiero publica `PagoAprobadoEvento` y se desentiende; comunicaciones escucha. Esto reproduce exactamente la Figura 6 de tu documento de requerimientos, pero además la hace cierta en el código.

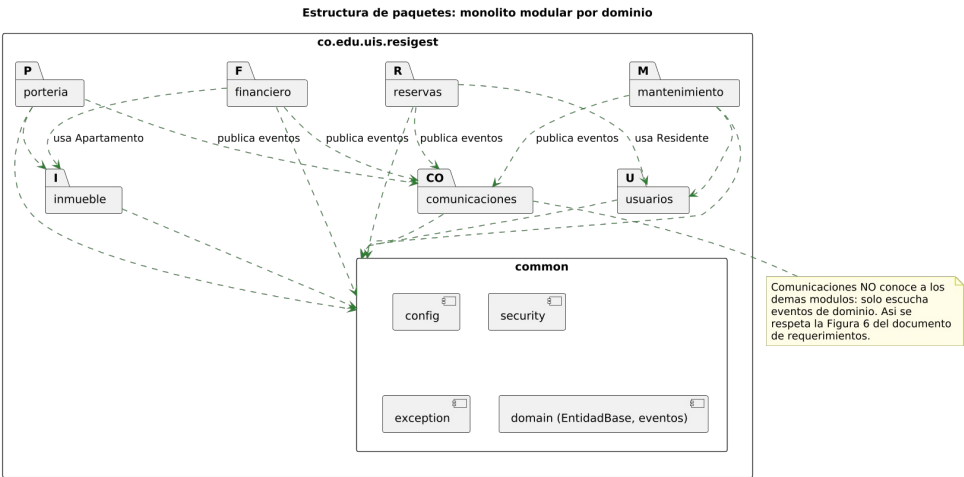


Figura 2. Módulos del monolito y sus dependencias permitidas.

2.4 Decisiones tomadas y sus alternativas

Decisión	Por qué	Qué se descartó
Módulos por dominio, no por capa técnica	Al abrir el paquete financiero ves todo lo del negocio junto; el código cambia por razones de negocio.	Paquetes controllers/, services/, entities/ globales: se vuelven inmanejables pasadas 30 clases.
Interfaz + implementación en servicios	Permite simular servicios en pruebas y documenta el contrato del módulo.	Clase concreta directa: válido en proyectos chicos, pero aquí pierdes claridad y facilidad de prueba.
DTOs con record	Inmutables, sin boilerplate, imposible que se filtre el hash de contraseña a la API.	Devolver entidades: genera ciclos infinitos de JSON y expone campos internos.
Flyway para el esquema	El esquema queda versionado en Git; tus compañeros levantan la base idéntica.	ddl -auto=update en producción: borra o rompe datos sin avisar.
JWT sin estado	El frontend en React guarda el token; el backend no mantiene sesión en memoria.	Sesión con cookie de servidor: complica el despliegue y el consumo desde React.
Eventos de dominio internos	Desacopla notificaciones con ApplicationEventPublisher, que ya viene en Spring.	Colas tipo RabbitMQ o Kafka: infraestructura extra sin beneficio para el alcance del curso.

3. Estructura de paquetes y carpetas

Este es el árbol completo del proyecto. No lo crees todo de una vez: crea cada paquete cuando llegues a él en la hoja de ruta. Tenerlo aquí sirve para que sepas siempre dónde va el archivo que estás por escribir.

```

resigest-backend/
├── pom.xml
├── docker-compose.yml
├── src/
│   └── main/
│       ├── java/co/edu/uis/resigest/
│       │   ├── ResigestApplication.java          <- clase con @SpringBootApplication
│       │   ├── common/
│       │   │   ├── config/      SecurityConfig, CorsConfig, OpenApiConfig, JpaAuditingConfig
│       │   │   ├── security/    JwtService, JwtAuthenticationFilter, UsuarioDetailsService,
│       │   │   │               UsuarioAutenticado
│       │   │   ├── exception/   GlobalExceptionHandler, ApiError, RecursoNoEncontradoException,
│       │   │   │               ReglaNegocioException, ConflictoException
│       │   │   ├── domain/      EntidadBase, EventoDominio
│       │   │   └── util/        PeriodoUtils, DineroUtils
│       │   ├── usuarios/
│       │   │   ├── domain/      Usuario, Residente, Administrador, Portero,
│       │   │   │               TecnicoMantenimiento, Rol, TipoResidente
│       │   │   ├── repository/  UsuarioRepository, ResidenteRepository
│       │   │   ├── service/     AuthService(+Impl), UsuarioService(+Impl)
│       │   │   ├── dto/         LoginRequest, TokenResponse, CrearUsuarioRequest, UsuarioDTO
│       │   │   ├── mapper/      UsuarioMapper
│       │   │   └── web/         AuthController, UsuarioController
│       │   ├── inmueble/
│       │   │   ├── domain/      Torre, Apartamento
│       │   │   ├── repository/  TorreRepository, ApartamentoRepository
│       │   │   ├── service/     ApartamentoService(+Impl)
│       │   │   ├── dto/         ApartamentoDTO, CrearApartamentoRequest
│       │   │   └── web/         ApartamentoController
│       │   ├── financiero/
│       │   │   ├── domain/      ObligacionFinanciera, Pago, PagoPasarela, PagoManual,
│       │   │   │               Comprobante, FacturaServicioComun, DetalleProrrrateo,
│       │   │   │               EstadoObligacion, EstadoPago, ConceptoCobro, TipoServicio
│       │   │   ├── repository/  ObligacionRepository, PagoRepository, FacturaServicioRepository
│       │   │   ├── service/     CarteraService(+Impl), PagoService(+Impl),
│       │   │   │               ProrrrateoService(+Impl), ComprobanteService(+Impl)
│       │   │   ├── gateway/     PasarelaPago (interfaz), WompiPasarelaAdapter,
│       │   │   │               SolicitudPago, RespuestaTransaccion
│       │   │   ├── pdf/         GeneradorComprobante (interfaz), GeneradorComprobantePdf
│       │   │   ├── event/       PagoAprobadoEvento, PagoRechazadoEvento
│       │   │   ├── dto/         ObligacionDTO, EstadoCuentaDTO, IniciarPagoRequest,
│       │   │   │               ReportarPagoManualRequest, ProrrrateoRequest
│       │   │   └── web/         ObligacionController, PagoController, WebhookController,
│       │   │               ProrrrateoController
│       │   ├── reservas/
│       │   │   ├── domain/      ZonaComun, Reserva, EstadoReserva
│       │   │   ├── repository/  ZonaComunRepository, ReservaRepository
│       │   │   ├── service/     ReservaService(+Impl), ZonaComunService(+Impl)
│       │   │   ├── validation/  ValidadorReserva (interfaz), ValidadorMorosidad,
│       │   │   │               ValidadorHorarioZona, ValidadorSolapamiento, ValidadorMaxHoras
│       │   │   ├── event/       ReservaResueltaEvento
│       │   │   ├── dto/         ReservaDTO, SolicitudReservaRequest, RechazoRequest
│       │   │   └── web/         ReservaController, ZonaComunController
│       │   ├── mantenimiento/
│       │   │   ├── domain/      Incidencia, EvidenciaIncidencia, EstadoIncidencia,
│       │   │   │               PrioridadIncidencia
│       │   │   ├── repository/  IncidenciaRepository
│       │   │   ├── service/     IncidenciaService(+Impl)
│       │   │   ├── storage/     AlmacenamientoArchivos (interfaz), AlmacenamientoLocal
│       │   │   ├── event/       IncidenciaAsignadaEvento
│       │   │   ├── dto/         IncidenciaDTO, CrearIncidenciaRequest, AsignarRequest
│       │   │   └── web/         IncidenciaController
│       │   ├── porteria/
│       │   │   ├── domain/      RegistroIngreso, Paquete, TipoIngreso, EstadoPaquete
│       │   │   ├── repository/  RegistroIngresoRepository, PaqueteRepository
│       │   │   ├── service/     PorteriaService(+Impl), PaqueteService(+Impl)
│       │   │   ├── event/       IngresoRegistradoEvento, PaqueteRecibidoEvento
│       │   │   ├── dto/         RegistroIngresoDTO, RegistrarIngresoRequest, PaqueteDTO
│       │   │   └── web/         PorteriaController, PaqueteController
│       │   └── comunicaciones/
│       │       ├── domain/      Comunicado, Notificacion, TipoNotificacion, TipoCanal
│       │       ├── repository/  ComunicadoRepository, NotificacionRepository
│       │       ├── service/     ComunicadoService(+Impl), NotificacionService(+Impl)
│       │       ├── channel/     CanalNotificacion (interfaz), CanalEnPlataforma,
│       │       │               CanalCorreo, CanalPush
│       │       └── listener/     ManejadorEventosNotificacion

```

```

├── dto/           ComunicadoDTO, CrearComunicadoRequest, NotificacionDTO
├── web/           ComunicadoController, NotificacionController
├── resources/
│   ├── application.yml           (config común)
│   ├── application-dev.yml       (base local, logs SQL activos)
│   ├── application-prod.yml      (variables de entorno, sin secretos en Git)
│   └── db/migration/
│       ├── V1_esquema_usuarios_inmueble.sql
│       ├── V2_esquema_financiero.sql
│       ├── V3_esquema_reservas.sql
│       ├── V4_esquema_mantenimiento_porteria.sql
│       ├── V5_esquema_comunicaciones.sql
│       └── V6_datos_iniciales.sql
└── test/java/co/edu/uis/resigest/ (espeja la estructura de main)

```

Sobre el paquete common. Solo entra ahí lo que usan *tres o más* módulos. Si algo lo usan dos, vive en el módulo dueño y el otro lo importa. *common* tiende a convertirse en un basurero; vigílalo.

Anatomía de un módulo

Los siete módulos repiten exactamente el mismo esqueleto. Interioriza esta figura y ya sabes construir cualquiera de ellos.

Anatomía de un modulo (plantilla que se repite en los 7 modulos)

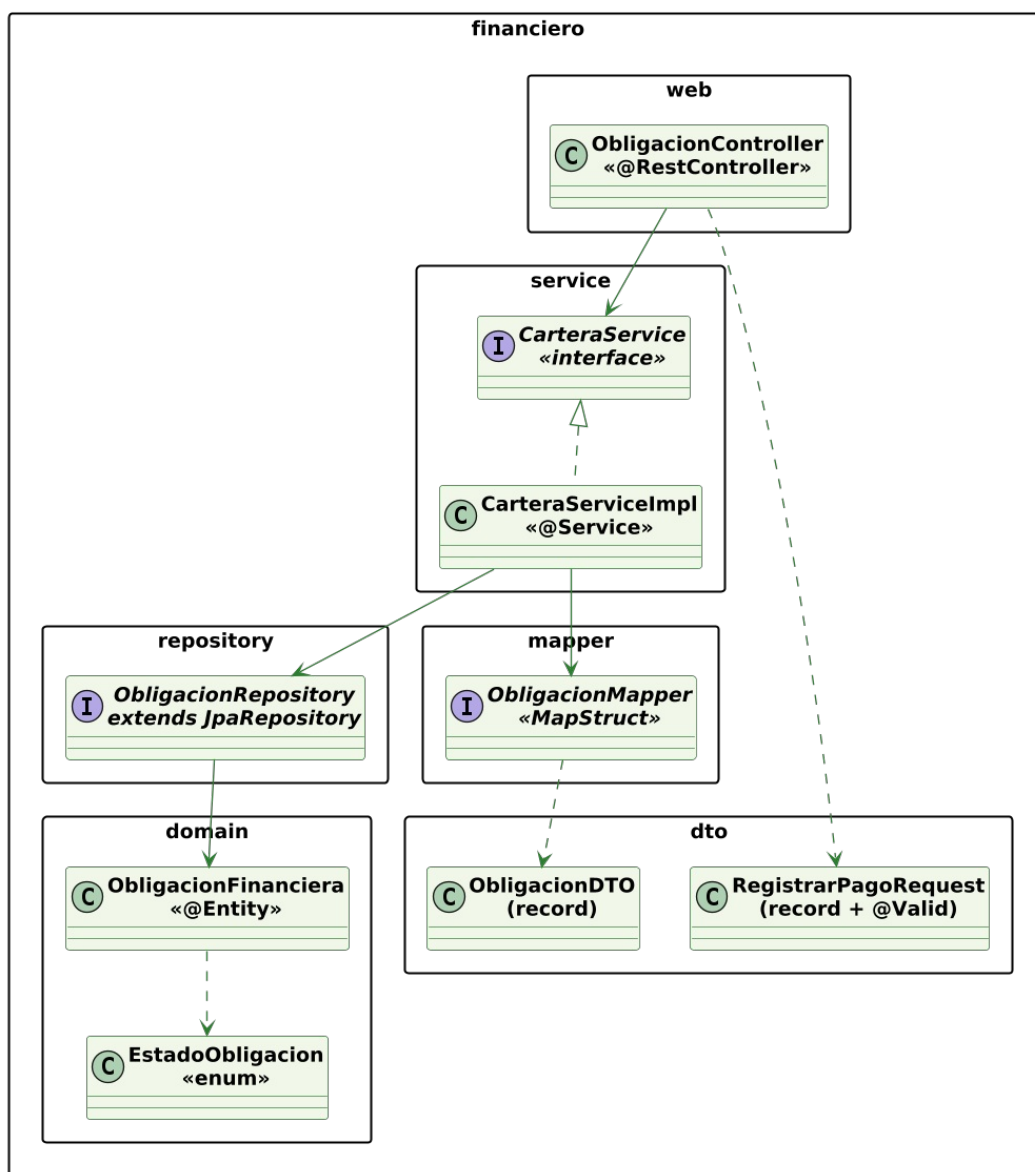


Figura 3. Plantilla de un módulo: web → service → repository → domain, con DTO y mapper laterales.

4. Convenciones de nombres y responsabilidades por tipo de clase

Tipo	Convención	Responsabilidad única
Entidad	Sustantivo singular: <code>Reserva</code>	Representar un concepto del negocio y proteger sus propias reglas internas (una reserva no puede aprobarse dos veces).
Enum	<code>EstadoReserva</code> , <code>Rol</code>	Enumerar valores cerrados. Nunca uses <code>String</code> para un estado.
Repositorio	<code>ReservaRepository</code>	Acceso a datos. Es una interfaz que extiende <code>JpaRepository</code> ; no se implementa a mano.
Servicio (interfaz)	<code>ReservaService</code>	Declarar el contrato del caso de uso: qué se puede hacer con reservas.
Servicio (impl)	<code>ReservaServiceImpl</code>	Implementar las reglas, coordinar repositorios y publicar eventos.
Controlador	<code>ReservaController</code>	Traducir HTTP ↔ llamadas al servicio. Idealmente ningún método pasa de 5 líneas.
DTO de entrada	<code>SolicitudReservaRequest</code>	Recibir datos del frontend ya validados con anotaciones de <code>jakarta.validation</code> .
DTO de salida	<code>ReservaDTO</code>	Exponer solo lo que el frontend necesita ver.
Mapper	<code>ReservaMapper</code>	Convertir entidad ↔ DTO. Con <code>MapStruct</code> es una interfaz anotada.
Excepción	<code>ReglaNegocioException</code>	Señalar una violación de negocio; el <code>GlobalExceptionHandler</code> la traduce a HTTP.
Evento	<code>ReservaResueltaEvento</code>	Anunciar que algo relevante ocurrió, en pasado.

Cuatro reglas de oro

- Todo en español, menos las palabras del framework.** Tus clases y campos en español (`saldoPendiente`), las anotaciones y métodos heredados en inglés. Es coherente con la documentación que ya escribió tu equipo.
- Nunca devuelvas una entidad desde un controlador.** Siempre DTO. Sin excepciones.
- Los métodos que cambian estado van en la entidad, no en el servicio.** `reserva.aprobar(admin)` es mejor que `reserva.setEstado(APROBADA)` desde el servicio: la entidad protege su regla de que solo una reserva PENDIENTE puede aprobarse.
- Dinero siempre en `BigDecimal`.** Nunca `double` ni `float`: producen errores de centavos que en un módulo de cartera son inaceptables.

5. Modelo de dominio por módulo

Las tablas de esta sección son tu lista de trabajo. Cada fila es una clase que debes crear, con sus atributos y sus métodos. Los métodos marcados `negocio` contienen reglas: son los que debes probar con JUnit. Los demás son consultas o accesos simples.

5.1 Módulo `common`: la base de todo

Clase	Atributos / campos	Métodos y notas
EntidadBase clase abstracta, <code>@MappedSuperclass</code>	<code>Long id (@Id @GeneratedValue)</code> <code>LocalDateTime fechaCreacion</code> <code>LocalDateTime fechaActualizacion</code>	No tiene métodos propios más allá de <code>equals</code> y <code>hashCode</code> basados en el id. Todas las entidades del sistema la extienden, de modo que nunca repites esos tres campos. Se anota con <code>@EntityListeners(AuditingEntityListener.class)</code> y las fechas se llenan solas.
EventoDominio clase abstracta	<code>LocalDateTime ocurridoEn</code>	<code>abstract String descripcion()</code> <code>negocio</code> — cada evento concreto explica qué pasó, en pasado.
ApiError <code>record</code>	<code>LocalDateTime timestamp</code> , <code>int estado</code> , <code>String codigo</code> , <code>String mensaje</code> , <code>Map<String,String> erroresCampo</code>	Formato único de error para todo el API. El frontend de tus compañeros solo tiene que aprender esta forma una vez.
RecursoNoEncontradoException	hereda de <code>RuntimeException</code>	Se lanza cuando un <code>findById</code> viene vacío. El handler la traduce a 404 .
ReglaNegocioException	hereda de <code>RuntimeException</code>	Violación de una regla: residente moroso, zona ocupada, transición de estado inválida. Se traduce a 409 o 422 .
ConflictoException	hereda de <code>RuntimeException</code>	Choque de concurrencia (dos reservas al mismo tiempo). Se traduce a 409 .
GlobalExceptionHandler <code>@RestControllerAdvice</code>	—	Un método <code>@ExceptionHandler</code> por cada excepción anterior, más uno para <code>MethodArgumentNotValidException</code> (errores de validación de DTO) y uno genérico de respaldo. Es la única clase del proyecto que decide códigos HTTP de error.

5.2 Módulos `usuarios` e `inmueble`

Modulo Usuarios / Inmueble - herencia JOINED

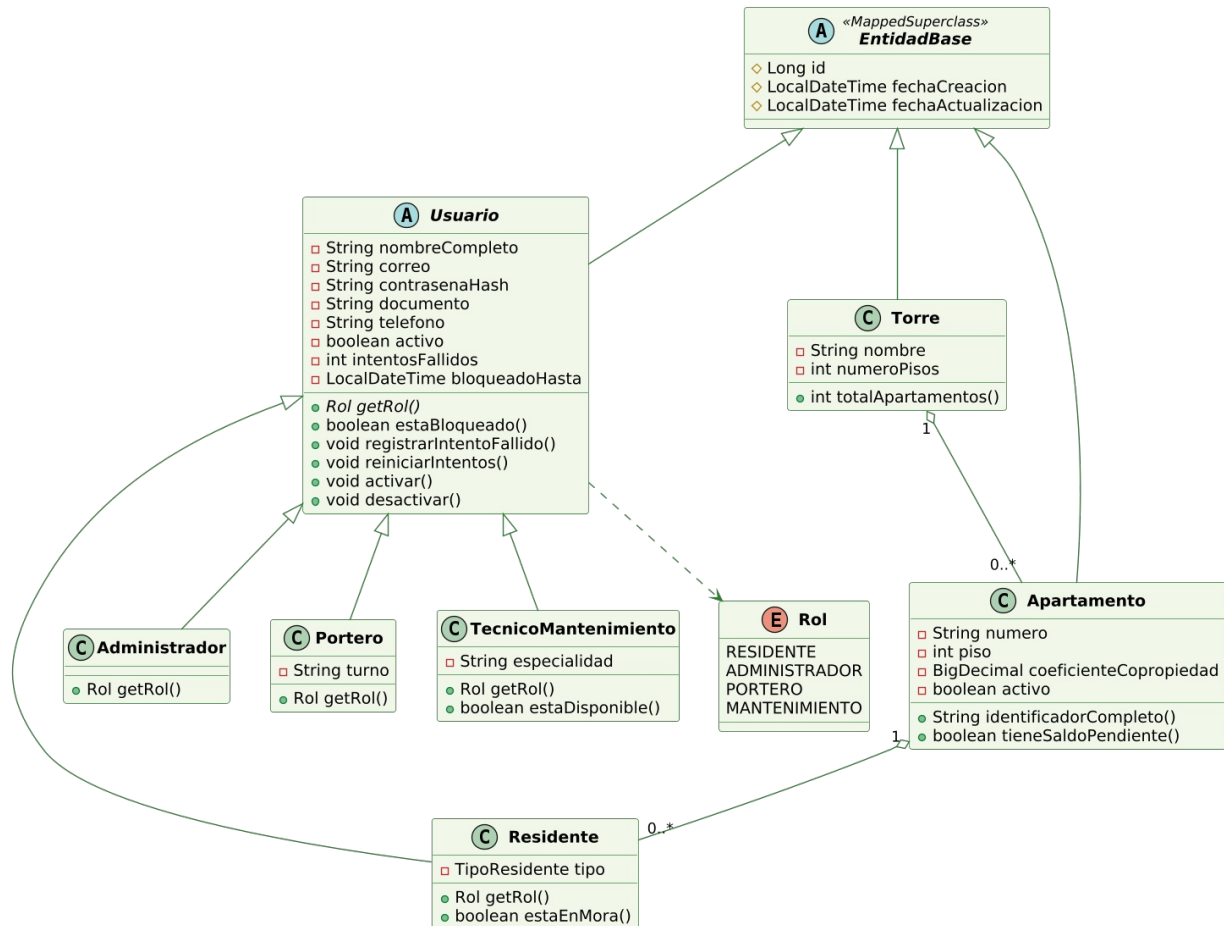


Figura 4. Jerarquía de usuarios con herencia JOINED y su relación con Torre y Apartamento.

Clase	Atributos	Métodos
Usuario abstracta, @Entity, @Inheritance(JOINED)	String nombreCompleto String correo (único) String contrasenaHash String documento String telefono boolean activo int intentosFallidos LocalDateTime bloqueadoHasta	abstract Rol getRol() boolean estaBloqueado() negocio — compara bloqueadoHasta con el ahora. void registrarIntentoFallido() negocio — al llegar a 5, fija bloqueo por 15 minutos (RF01). void reiniciarIntentos() void activar() / void desactivar()
Residente extends Usuario	Apartamento apartamento (@ManyToOne) TipoResidente tipo	Rol getRol() → RESIDENTE boolean estaEnMora() negocio — precondition de RF03; se apoya en la cartera del apartamento.
Administrador extends Usuario	—	Rol getRol() → ADMINISTRADOR
Portero extends Usuario	String turno	Rol getRol() → PORTERO
TecnicoMantenimiento extends Usuario	String especialidad	Rol getRol() → MANTENIMIENTO boolean estaDisponible()
Torre	String nombre, int numeroPisos, List<Apartamento>	int totalApartamentos()
Apartamento	String numero, int piso, BigDecimal coeficienteCopropiedad, Torre torre, boolean activo	String identificadorCompleto() — “Torre 2 - Apto 304”. boolean tieneSaldoPendiente() negocio

Repositorios y servicios del módulo

Interfaz / clase	Métodos que debes declarar
UsuarioRepository extends JpaRepository<Usuario, Long>	Optional<Usuario> findByCorreo(String correo) boolean existsByCorreo(String correo) List<Usuario> findByActivoTrue() Spring genera la implementación a partir del nombre del método. No escribes SQL.
ResidenteRepository	List<Residente> findByApartamentoId(Long id) Optional<Residente> findByCorreo(String correo)

ApartamentoRepository	<code>Optional<Apartamento> findByTorreIdAndNumero(Long torreId, String numero)</code> <code>@Query para BigDecimal sumaCoeficientes()</code> — se usa en la validación de RF07.
AuthService (interfaz)	<code>TokenResponse autenticar(LoginRequest req)</code> negocio <code>void solicitarRecuperacion(String correo)</code> <code>void restablecerContraseña(String token, String nuevaContraseña)</code>
AuthServiceImpl	Implementa lo anterior: busca por correo, verifica bloqueo, compara con <code>PasswordEncoder.matches</code> , registra intento fallido o reinicia contador, y delega en <code>JwtService</code> la emisión del token.
UsuarioService (interfaz)	<code>UsuarioDTO registrar(CrearUsuarioRequest req)</code> negocio (CU05) <code>UsuarioDTO obtener(Long id)</code> <code>Page<UsuarioDTO> listar(Rol rol, Pageable pageable)</code> <code>void desactivar(Long id)</code>
JwtService	<code>String generarToken(Usuario u)</code> , <code>String extraerCorreo(String token)</code> , <code>boolean esValido(String token)</code> . Incluye el rol como <i>claim</i> para que el frontend sepa qué menú pintar.

Por qué herencia JOINED y no una sola tabla. Con JOINED, Hibernate crea una tabla `usuario` con lo común y una tabla por subclase con sus campos propios. Así la columna `apartamento_id` puede ser NOT NULL en la tabla de residentes sin obligar a los porteros a tener apartamento. Con SINGLE_TABLE tendrías una sola tabla llena de columnas nulas. El costo es un *join* extra por consulta, irrelevante con el volumen de un edificio.

5.3 Módulo financiero

Es el módulo más complejo del sistema y el que más nota puede darte, porque concentra reglas reales: saldos, estados, dos formas de pago y un cálculo proporcional.

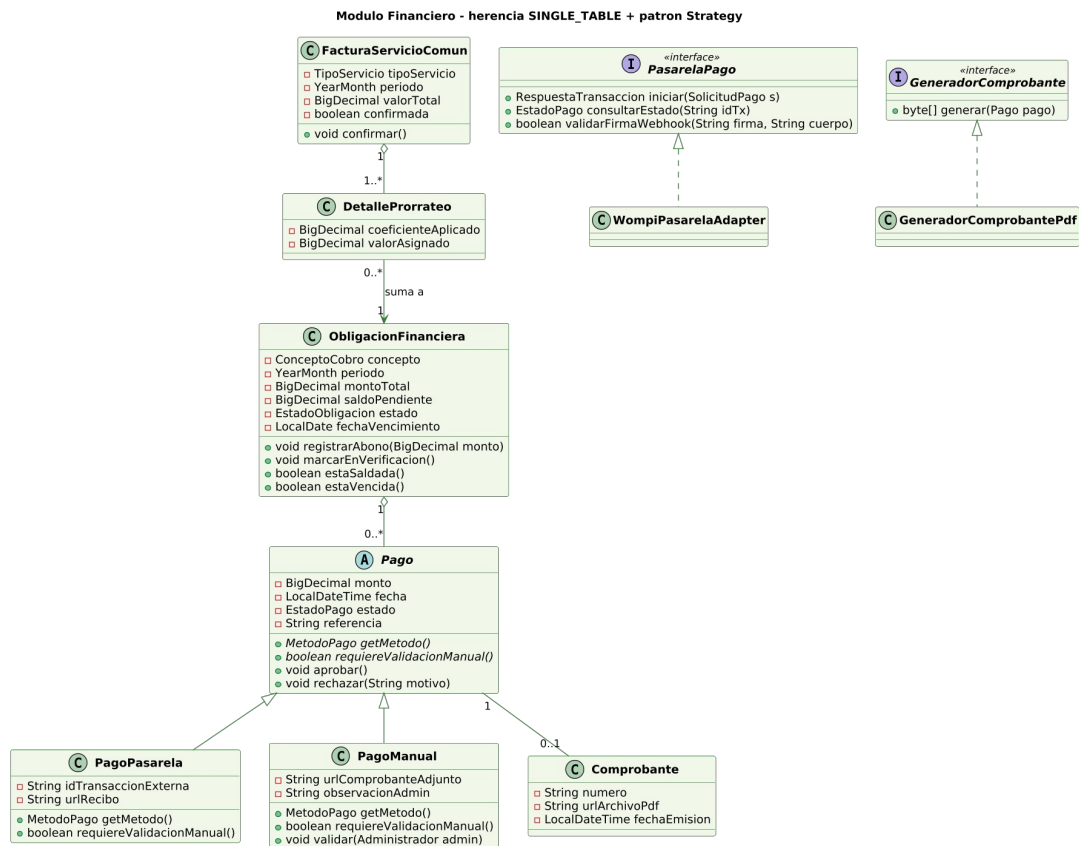


Figura 5. Módulo financiero: jerarquía de pagos y puertos hacia la pasarela y el generador de PDF.

Clase	Atributos	Métodos
ObligacionFinanciera	Apartamento apartamento ConceptoCobro concepto YearMonth periodo BigDecimal montoTotal BigDecimal saldoPendiente EstadoObligacion estado LocalDate fechaVencimiento List<Pago> pagos	void registrarAbono(BigDecimal monto) negocio — resta del saldo y, si llega a cero, pasa a PAGADA. Rechaza montos negativos o mayores al saldo. void marcarEnVerificacion() negocio boolean estaSaldada() boolean estaVencida() negocio
Pago abstracta, @Inheritance(SINGLE_TABLE), @DiscriminatorColumn("metodo")	BigDecimal monto LocalDateTime fecha EstadoPago estado String referencia ObligacionFinanciera obligacion	abstract MetodoPago getMetodo() abstract boolean requiereValidacionManual() void aprobar() negocio — solo desde INICIADO o EN_VERIFICACION. void rechazar(String motivo) negocio
PagoPasarela extends Pago	String idTransaccionExterna String urlRecibo	getMetodo() → WOMPI; requiereValidacionManual() → false.
PagoManual extends Pago	String urlComprobanteAdjunto String observacionAdmin Administrador validadoPor	getMetodo() → TRANSFERENCIA; requiereValidacionManual() → true. void validar(Administrador admin) negocio (CU06)
Comprobante	String numero, String urlArchivoPdf, LocalDateTime fechaEmision, Pago pago	Se crea solo cuando el pago quedó aprobado (criterio de aceptación de RF02).
FacturaServicioComun	TipoServicio tipoServicio YearMonth periodo BigDecimal valorTotal boolean confirmada List<DetalleProrratio> detalles	void confirmar() negocio — una vez confirmada no se puede volver a prorratear.
DetalleProrratio	Apartamento apartamento, BigDecimal coeficienteAplicado, BigDecimal valorAsignado	Guarda el coeficiente <i>del momento del cálculo</i> , no una referencia viva: si mañana cambian los coeficientes, el histórico no se altera.

Servicios e interfaces del módulo financiero

Interfaz / clase	Métodos
CarteraService	EstadoCuentaDTO consultarEstadoCuenta(Long apartamentoId, YearMonth periodo) (CU11) List<ObligacionDTO> listarPendientes(Long apartamentoId) ObligacionDTO generarObligacionMensual(Long apartamentoId, YearMonth periodo) negocio void marcarVencidas() negocio — tarea programada con @Scheduled.
PagoService	PagoIniciadoDTO iniciarPagoPasarela(IniciarPagoRequest req) negocio (CU02) PagoDTO reportarPagoManual(ReportarPagoManualRequest req) negocio void procesarWebhook(String firma, String cuerpo) negocio PagoDTO validarPagoManual(Long pagoId, boolean aprobado, String observacion) negocio (CU06) List<PagoDTO> listarPendientesDeValidacion()
ProrratioService	ProrratioPreviewDTO calcular(ProrratioRequest req) negocio (CU10) — solo calcula, no guarda. void confirmar(Long facturaId) negocio — persiste los detalles y suma a cada obligación. Regla de RF07: si la suma de coeficientes $\neq$ 100 %, lanza ReglaNegocioException y no ejecuta nada.
PasarelaPago interfaz (puerto)	RespuestaTransaccion iniciar(SolicitudPago s) EstadoPago consultarEstado(String idTx) boolean validarFirmaWebhook(String firma, String cuerpo) Implementación: WompiPasarelaAdapter. En pruebas, un PasarelaSimulada que siempre aprueba: así avanza sin depender de credenciales reales.
GeneradorComprobante interfaz	byte[] generar(Pago pago) — implementada por GeneradorComprobantePdf.

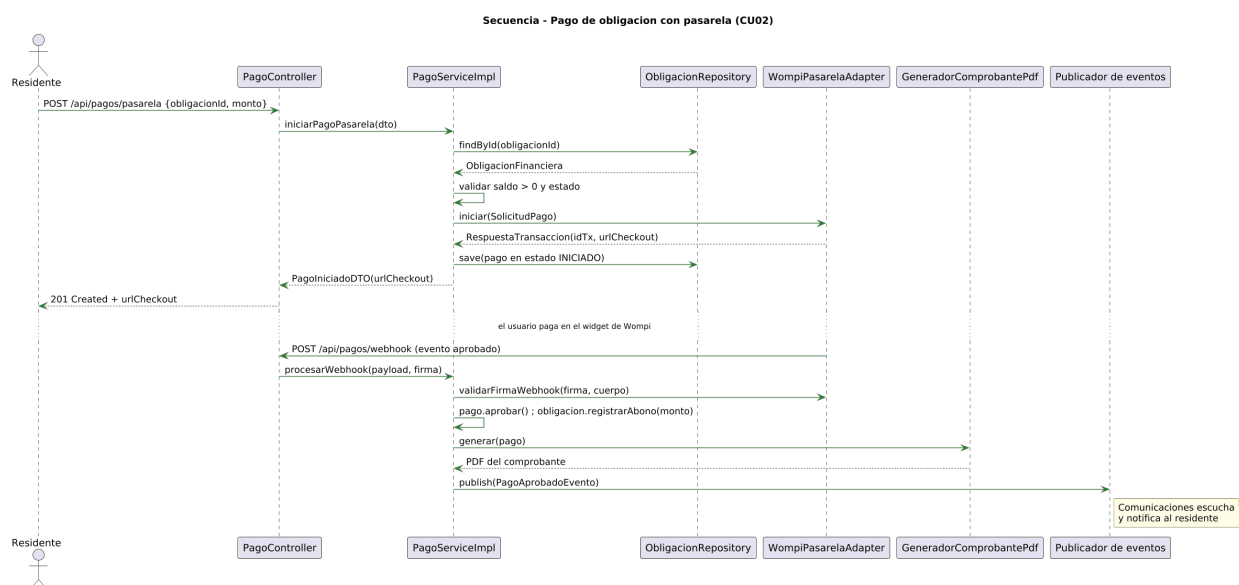


Figura 6. Secuencia del pago con pasarela: nota que el webhook es un segundo viaje, independiente del primero.

El punto que más se equivoca en este módulo: creer que el pago termina cuando el usuario vuelve del widget de Wompi. No es así. La confirmación fiable llega por **webhook**, en una petición aparte que hace la pasarela a tu servidor. Por eso **PagoService** tiene dos métodos separados y el endpoint del webhook es público (pero con validación de firma). Diseña esto desde el principio o tendrás que rehacerlo.

5.4 Módulo reservas

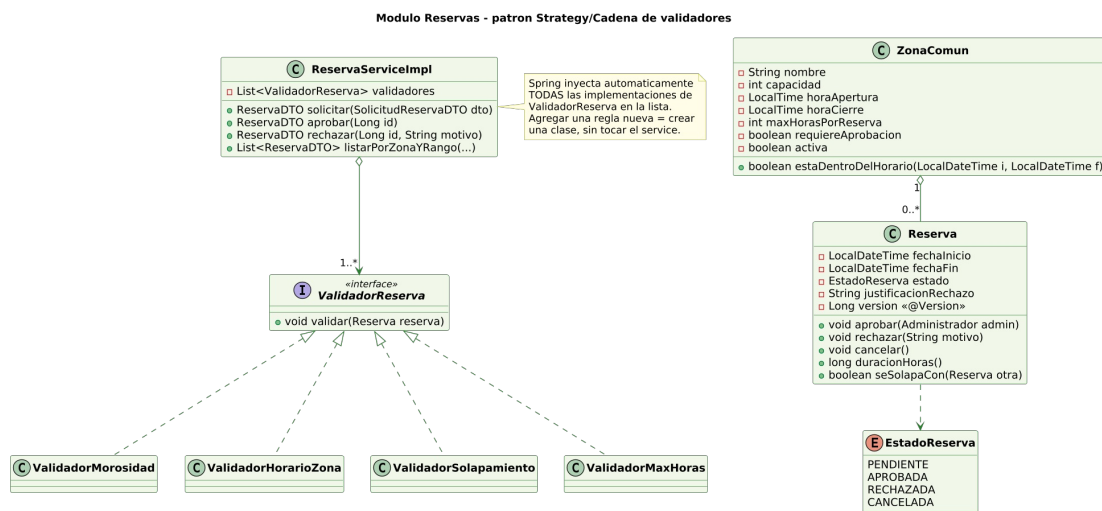


Figura 7. Reservas: la lista de validadores permite agregar reglas sin modificar el servicio.

Clase	Atributos	Métodos
ZonaComun	String nombre, int capacidad, LocalTime horaApertura, LocalTime horaCierre, int maxHorasPorReserva, boolean requiereAprobacion, boolean activa	boolean estaDentroDelHorario(LocalDateTime inicio, LocalDateTime fin) negocio
Reserva	ZonaComun zona, Residente solicitante, LocalDateTime fechaInicio, LocalDateTime fechaFin, EstadoReserva estado, String justificacionRechazo, Administrador resueltaPor, @Version Long version	void aprobar(Administrador admin) negocio void rechazar(String motivo) negocio — exige motivo no vacío (RF04). void cancelar() negocio long duracionHoras() boolean seSolapaCon(Reserva otra) negocio
ValidadorReserva interfaz	—	void validar(Reserva reserva) — lanza ReglaNegocioException si no cumple.
ValidadorMorosidad	—	Precondición de RF03: el residente no puede tener obligaciones vencidas.
ValidadorHorarioZona	—	La franja pedida debe caer entre apertura y cierre.
ValidadorMaxHoras	—	Flujo de validación A del CU03: no exceder el máximo de horas.
ValidadorSolapamiento	—	Consulta al repositorio si existe otra reserva viva en esa franja.
ReservaService interfaz	—	ReservaDTO solicitar(SolicitudReservaRequest req) negocio (CU03) ReservaDTO aprobar(Long id) negocio (CU07) ReservaDTO rechazar(Long id, String motivo) negocio void cancelar(Long id) List<ReservaDTO> consultarCalendario(Long zonaId, LocalDate desde, LocalDate hasta) List<ReservaDTO> listarPendientes()
ReservaRepository	—	boolean existsSolapamiento(Long zonaId, LocalDateTime inicio, LocalDateTime fin) con @Query; List<Reserva> findByEstado(EstadoReserva e); List<Reserva> findByZonaIdAndFechaInicioBetween(...).

Cómo funciona el bloqueo optimista que menciona tu RF03. El campo @Version hace que Hibernate incluya la versión en el UPDATE. Si dos residentes reservan la misma franja en el mismo instante, el segundo UPDATE no afecta ninguna fila y Spring lanza OptimisticLockingFailureException. Tú la capturas en el servicio y la conviertes en ConflictException → HTTP 409. Complemento recomendado: una restricción UNIQUE parcial en base de datos para reservas aprobadas de la misma zona y franja, porque el bloqueo optimista solo protege filas ya existentes.

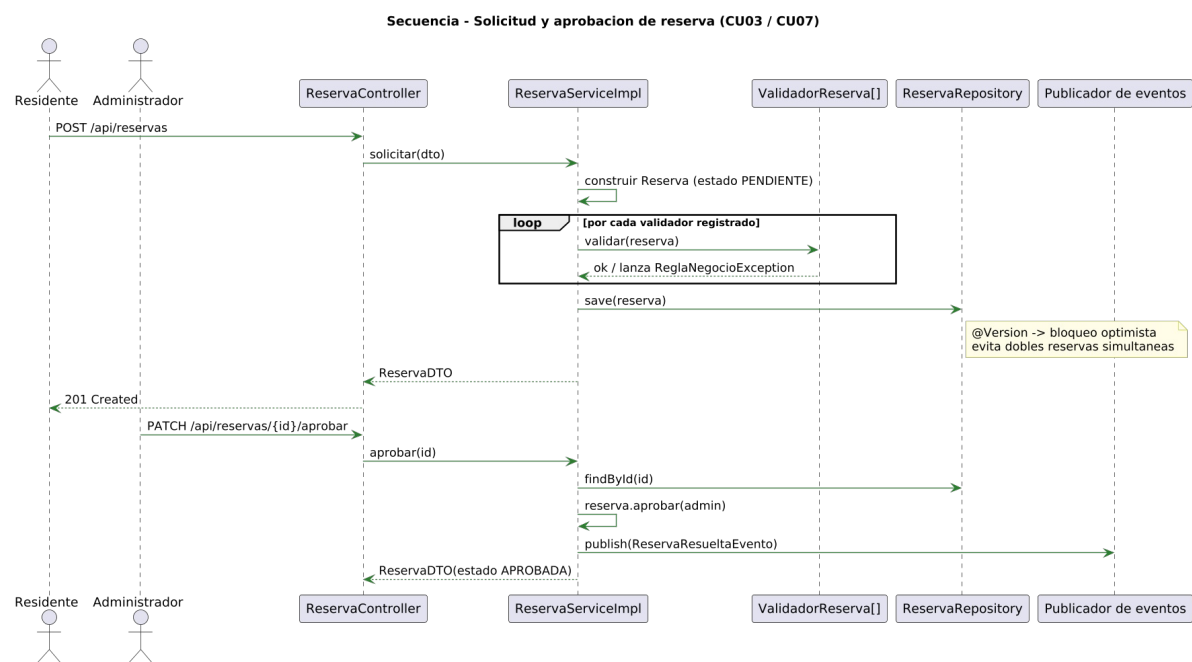


Figura 8. Secuencia de solicitud y aprobación de reserva.

5.5 y 5.6 Módulos mantenimiento y porteria

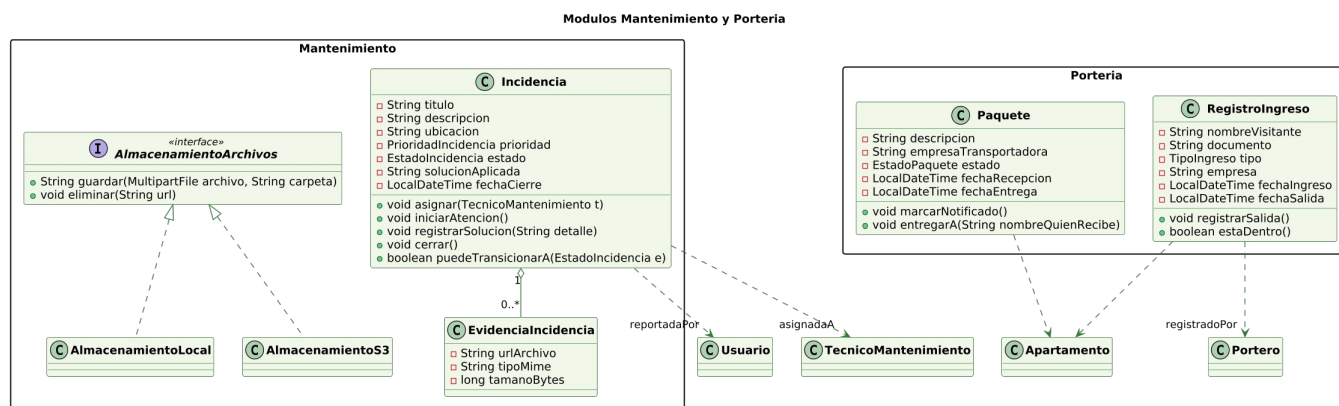


Figura 9. Incidencias con evidencias y almacenamiento intercambiable; registros de portería y paquetería.

Clase	Atributos	Métodos
Incidencia	String titulo, String descripcion, String ubicacion, PrioridadIncidencia prioridad, EstadoIncidencia estado, Usuario reportadaPor, TecnicoMantenimiento asignadaA, String solucionAplicada, LocalDateTime fechaCierre, List<EvidenciaIncidencia> evidencias	void asignar(TecnicoMantenimiento t) negocio — solo si está ABIERTA. void iniciarAtencion() negocio void registrarSolucion(String detalle) negocio — obligatoria para cerrar (CU09). void cerrar() negocio boolean puedeTransicionarA(EstadoIncidencia e) negocio
EvidenciaIncidencia	String urlArchivo, String tipoMime, long tamanoBytes	Validación de RF06: rechazar archivos de más de 5 MB con mensaje claro.
AlmacenamientoArchivos interfaz	—	String guardar(MultipartFile archivo, String carpeta), void eliminar(String url). Implementa AlmacenamientoLocal ahora y, si algún día despliegan en la nube, agregas otra implementación sin tocar el servicio.
IncidenciaService	—	IncidenciaDTO reportar(CrearIncidenciaRequest req) (CU08) IncidenciaDTO asignar(Long id, Long tecnicoId) (CU09) IncidenciaDTO resolver(Long id, String solucion) List<IncidenciaDTO> listarPorEstado(EstadoIncidencia e) List<IncidenciaDTO> listarAsignadasA(Long tecnicoId)
RegistroIngreso	String nombreVisitante, String documento, TipoIngreso tipo, String empresa, Apartamento apartamentoDestino, Portero registradoPor, LocalDateTime fechaIngreso, LocalDateTime fechaSalida	void registrarSalida() negocio boolean estaDentro() RF05 exige bitácora inmutable: no expongas endpoints de edición ni de borrado.
Paquete	Apartamento apartamentoDestino, String empresaTransportadora, String descripcion, EstadoPaquete estado, LocalDateTime fechaRecepcion, LocalDateTime fechaEntrega, String entregadoA	void marcarNotificado() void entregarA(String nombreQuienRecibe) negocio
PorteriaService	—	RegistroIngresoDTO registrarIngreso(RegistrarIngresoRequest req) negocio (CU04) void registrarSalida(Long id) Page<RegistroIngresoDTO> consultarBitacora(LocalDate desde, LocalDate hasta, Pageable p)

5.7 Módulo comunicaciones

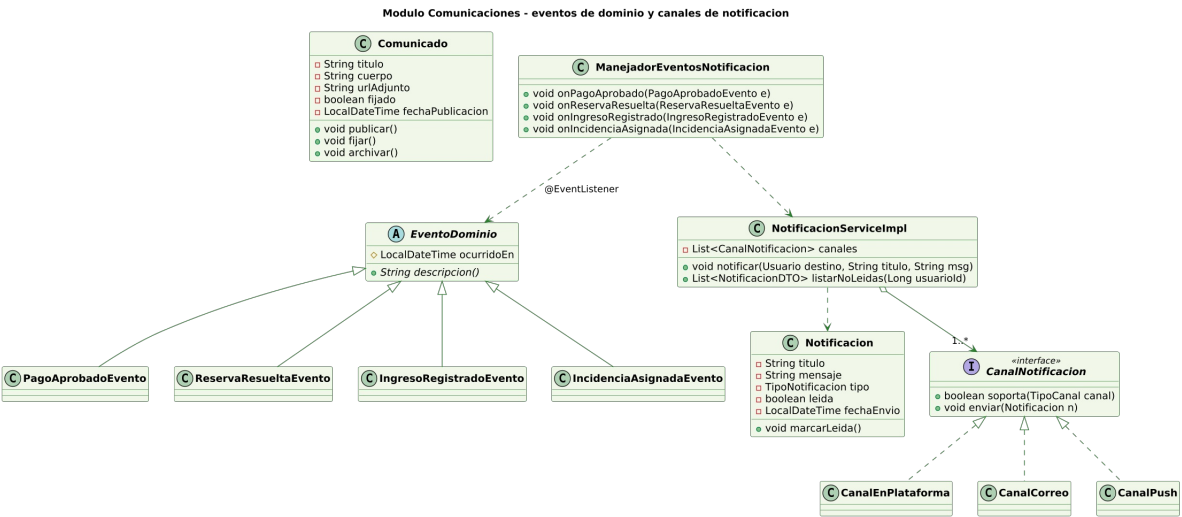


Figura 10. Eventos de dominio y canales de notificación intercambiables.

Clase	Atributos	Métodos
Comunicado	String titulo, String cuerpo, String urlAdjunto, boolean fijado, Administrador autor, LocalDateTime fechaPublicacion	void publicar() negocio (CU12) void fijar() void archivar()
Notificacion	Usuario destinatario, String titulo, String mensaje, TipoNotificacion tipo, boolean leida, LocalDateTime fechaEnvio, String referencia	void marcarLeida()
CanalNotificacion interfaz	—	boolean soporta(TipoCanal canal), void enviar(Notificacion n). Implementaciones: CanalEnPlataforma (guarda en base), CanalCorreo (JavaMailSender), CanalPush (opcional, déjalo para el final).
NotificacionServiceImpl	List<CanalNotificacion> canales (inyectada por Spring)	void notificar(Usuario destino, String titulo, String mensaje, TipoCanal... canales) List<NotificacionDTO> listarNoLeidas(Long usuarioId) void marcarLeida(Long id)
ManejadorEventosNotificacion @Component con @EventListener	—	Un método por evento: onPagoAprobado, onReservaResuelta, onIngresoRegistrado, onIncidenciaAsignada. Cada uno arma el mensaje y llama al servicio de notificaciones. Anótalos con @TransactionalEventListener(phase = AFTER_COMMIT) : así no envías un correo de “pago aprobado” si la transacción termina revertida — un detalle que muy pocos proyectos de curso cuidan.

Estados del sistema

Estos tres diagramas resumen las reglas de transición que tus métodos negocio deben hacer cumplir. Cualquier intento de transición que no aparezca aquí debe lanzar excepción.

Diagramas de estados de las entidades clave

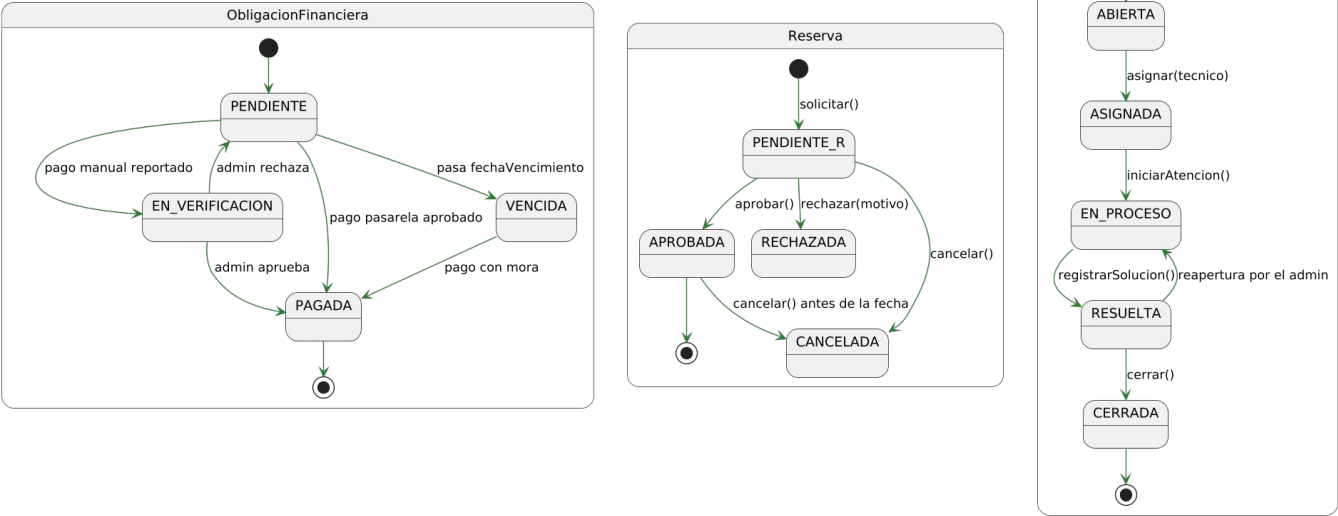


Figura 11. Máquinas de estado de obligación, reserva e incidencia.

6. Dónde aplicar herencia y dónde aplicar interfaces

Esta pregunta la hiciste explícitamente, y merece una respuesta con criterio, no una lista. La regla práctica: **hereda cuando dos cosas son lo mismo con variantes; usa interfaz cuando dos cosas hacen lo mismo de maneras distintas.**

6.1 Herencia (extends): úsala en tres lugares y en ninguno más

Jerarquía	Estrategia JPA	Justificación
EntidadBase → todas las entidades	@MappedSuperclass	No es una jerarquía de negocio: es reutilización de tres campos técnicos. No genera tabla propia.
Usuario → Residente, Administrador, Portero, TecnicoMantenimiento	@Inheritance(JOINED)	Comparten identidad y credenciales, difieren en datos propios. El polimorfismo aparece en <code>getRol()</code> y permite que Spring Security trate a todos igual al autenticar.
Pago → PagoPasarela, PagoManual	@Inheritance(SINGLE_TABLE)	Son el mismo hecho contable con procedencia distinta. Como comparten casi todos los campos y siempre se consultan juntos ("historial de pagos"), una sola tabla con columna discriminadora es lo correcto.
EventoDominio → los cuatro eventos	No es entidad	Objetos en memoria, nunca se guardan. La clase base solo aporta la marca de tiempo.

Herencia que NO debes hacer. No crees `ServicioBase` del que hereden todos tus servicios, ni `ControladorBase`. Es la tentación clásica de principiante y produce clases padre gigantes que nadie entiende. Para compartir comportamiento entre servicios, usa composición: inyecta un colaborador.

6.2 Interfaces: seis usos legítimos en este proyecto

Interfaz	Implementaciones	Qué problema resuelve
XxxRepository	Las genera Spring	Declaras qué consultas necesitas; el framework escribe el acceso a datos.
XxxService	XxxServiceImpl	Define el contrato del módulo y permite sustituirlo por un doble en las pruebas del controlador.
PasarelaPago	WompiPasarelaAdapter, PasarelaSimulada	Aísla un servicio externo del que no controlas la disponibilidad. Puedes desarrollar y probar sin credenciales de Wompi.
CanalNotificacion	En plataforma, correo, push	Agregar un canal nuevo no obliga a tocar el servicio: Spring inyecta todas las implementaciones en una lista.
ValidadorReserva	Cuatro validadores	Cada regla de RF03 vive en su propia clase, se prueba sola y se agrega sin modificar código existente.
AlmacenamientoArchivos	Local, y en el futuro nube	El servicio de incidencias no sabe si el archivo va a disco o a un bucket.

El patrón que se repite tres veces. `PasarelaPago`, `CanalNotificacion`, `ValidadorReserva` y `AlmacenamientoArchivos` son el mismo principio: *Strategy*. Si puedes explicar en la sustentación por qué usaste este patrón y qué pasaría si no lo hubieras usado, tienes buena parte de la nota de diseño asegurada.

7. Contratos REST

Define estos endpoints antes de programarlos y compártelos con quien haga el frontend: así ambos trabajan en paralelo. Convenciones: rutas en plural, sustantivos, sin verbos; el verbo lo pone el método HTTP.

Método	Ruta	Rol	Caso de uso
POST	/api/auth/login	público	CU01 — devuelve token JWT y rol
POST	/api/auth/recuperar	público	CU01 flujo alternativo A
POST	/api/usuarios	ADMIN	CU05 — registrar usuario
GET	/api/usuarios?rol=&page=	ADMIN	Listado paginado
GET	/api/usuarios/me	autenticado	Perfil propio para el dashboard
POST	/api/apartamentos	ADMIN	Alta de apartamento y coeficiente
GET	/api/apartamentos/{id}/estado-cuenta	ADMIN, RESIDENTE dueño	CU11
GET	/api/obligaciones?apartmentId=&estado=	ADMIN, RESIDENTE	RF02
POST	/api/pagos/pasarela	RESIDENTE	CU02 — inicia transacción
POST	/api/pagos/manual	RESIDENTE	CU02 flujo alternativo — adjunta comprobante
POST	/api/pagos/webhook	público + firma	Confirmación asíncrona de la pasarela
GET	/api/pagos/pendientes-validacion	ADMIN	CU06
PATCH	/api/pagos/{id}/validacion	ADMIN	CU06 — aprobar o rechazar
GET	/api/pagos/{id}/comprobante	RESIDENTE dueño, ADMIN	Descarga del PDF
POST	/api/servicios/prorrato/preview	ADMIN	CU10 — cálculo sin persistir
POST	/api/servicios/prorrato/confirmar	ADMIN	CU10 — aplica a las obligaciones
GET	/api/zonas-comunes	autenticado	RF03
GET	/api/reservas?zonalId=&desde=&hasta=	autenticado	Calendario
POST	/api/reservas	RESIDENTE	CU03
PATCH	/api/reservas/{id}/aprobar	ADMIN	CU07
PATCH	/api/reservas/{id}/rechazar	ADMIN	CU07 — exige justificación
DELETE	/api/reservas/{id}	RESIDENTE dueño	Cancelación
POST	/api/incidencias	RESIDENTE, PORTERO	CU08
POST	/api/incidencias/{id}/evidencias	RESIDENTE	RF06 — multipart, máx. 5 MB
PATCH	/api/incidencias/{id}/asignacion	ADMIN	CU09
PATCH	/api/incidencias/{id}/solucion	MANTENIMIENTO	CU09
POST	/api/porteria/ingresos	PORTERO	CU04
PATCH	/api/porteria/ingresos/{id}/salida	PORTERO	RF05
GET	/api/porteria/bitacora?desde=&hasta=	ADMIN	RF05 — consulta histórica
POST	/api/paquetes	PORTERO	Recepción de paquetería
PATCH	/api/paquetes/{id}/entrega	PORTERO	Entrega al residente
POST	/api/comunicados	ADMIN	CU12
GET	/api/comunicados?fijados=true	autenticado	RF08 — cartelera
GET	/api/notificaciones/no-leidas	autenticado	Campana del dashboard

Códigos de respuesta que debes usar de forma consistente

Código	Cuándo
200	Consulta o actualización exitosa.
201	Creación exitosa; incluye cabecera <code>Location</code> con la URL del recurso creado.
204	Eliminación o acción sin cuerpo de respuesta.
400	DTO mal formado o validación de formato fallida.
401	Sin token o token vencido.
403	Token válido pero rol insuficiente (un residente intentando aprobar una reserva).

404	El recurso no existe.
409	Conflicto: zona ya reservada, correo duplicado, choque de concurrencia.
422	Regla de negocio violada: residente moroso, coeficientes que no suman 100 %.

8. Seguridad: autenticación JWT y matriz de permisos

8.1 Piezas y flujo

Clase	Rol en el flujo
SecurityConfig	@Configuration @EnableWebSecurity @EnableMethodSecurity. Declara el SecurityFilterChain: sesión sin estado, rutas públicas, y registra el filtro JWT antes del filtro de usuario y contraseña. Aquí también declaras el bean BCryptPasswordEncoder.
JwtService	Genera y valida el token; incluye correo, rol y expiración (sugerido: 8 horas).
JwtAuthenticationFilter	Extiende OncePerRequestFilter. Lee la cabecera Authorization: Bearer ..., valida y coloca la autenticación en el SecurityContextHolder.
UsuarioDetailsService	Implementa UserDetailsService: busca el usuario por correo y lo adapta al modelo de Spring Security, mapeando getRol() a un GrantedAuthority con prefijo ROLE_.

8.2 Matriz de permisos por módulo

Recurso	Residente	Administrador	Portero	Mantenimiento
Usuarios y apartamentos	solo su perfil	total	—	solo su perfil
Cartera y pagos	lectura y pago de su apto	total + validación manual	—	—
Prorrates de servicios	lectura del detalle	total	—	—
Reservas	crear y cancelar las propias	aprobar / rechazar	—	—
Incidencias	crear y ver las propias	ver todas y asignar	crear	ver asignadas y resolver
Portería	ver ingresos a su apto	bitácora completa	registrar	—
Comunicados	lectura	publicar	lectura	lectura

La trampa de seguridad más común en proyectos de curso: proteger por rol pero no por propietario. Un residente autenticado tiene rol RESIDENTE, así que @PreAuthorize("hasRole('RESIDENTE')") lo dejaría consultar **el estado de cuenta de cualquier apartamento** con solo cambiar el id en la URL. La verificación de que el recurso le pertenece va en el servicio, comparando el usuario autenticado contra el dueño del recurso. Escribe esa comprobación una vez, en un componente reusable, y aplícala en cartera, reservas, incidencias y portería.

9. Manejo de errores, validación y auditoría

9.1 Tres niveles de validación, y no se pisan

Nivel	Dónde	Qué valida
Formato	DTO de entrada, con <code>@NotBlank</code> , <code>@Email</code> , <code>@Positive</code> , <code>@Future</code>	Que los datos vengan bien escritos. Falla → 400.
Negocio	Servicio y métodos de la entidad	Que la operación tenga sentido: residente sin mora, zona libre, transición de estado permitida. Falla → 422 o 409.
Integridad	Base de datos: <code>UNIQUE</code> , <code>NOT NULL</code> , claves foráneas	Última línea de defensa contra condiciones de carrera. Falla → 409.

9.2 Auditoría

Con `@EnableJpaAuditing` y las anotaciones `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy` en `EntidadBase`, cada fila registra cuándo se creó, cuándo cambió y qué usuario la originó, sin que escribas una línea en los servicios. Para las entidades sensibles del RF05 (bitácora de portería) esto ya cubre el requisito de trazabilidad; para el módulo financiero conviene además nunca borrar pagos: si uno se anula, se marca `RECHAZADO`, no se elimina.

10. Base de datos, migraciones y mapeo

Configura `spring.jpa.hibernate.ddl-auto=validate` y deja que **Flyway** cree el esquema. Al principio parece más trabajo; a la tercera semana, cuando tus compañeros levanten la base con un comando y sin pedirte ayuda, entenderás por qué.

Migración	Contenido
V1_esquema_usuarios_inmueble.sql	torre, apartamento, usuario, residente, administrador, portero, tecnico_mantenimiento
V2_esquema_financiero.sql	obligacion_financiera, pago (tabla única con discriminador), comprobante, factura_servicio_comun, detalle_prorrateo
V3_esquema_reservas.sql	zona_comun, reserva (con columna <code>version</code> y restricción de solapamiento)
V4_esquema_mantenimiento_porteria.sql	incidencia, evidencia_incidencia, registro_ingreso, paquete
V5_esquema_comunicaciones.sql	comunicado, notificacion
V6_datos_iniciales.sql	Un administrador, dos torres, diez apartamentos con coeficientes que sumen 100 %, tres zonas comunes. Sin datos de prueba no puedes demostrar nada en la sustentación.

Índices que vas a necesitar

- `usuario(correo)` — único; se consulta en cada login.
- `obligacion_financiera(apartamento_id, periodo)` — el estado de cuenta lo usa siempre.
- `reserva(zona_comun_id, fecha_inicio, fecha_fin)` — la consulta de calendario y la de solapamiento.
- `registro_ingreso(fecha_ingreso)` — la bitácora se consulta por rango de fechas.
- `notificacion(destinatario_id, leida)` — la campana del dashboard.

Regla anti-desastre: una migración ya aplicada **jamás** se edita. Si te equivocaste en V2, corriges con una V7. Editar una migración vieja rompe la base de todos tus compañeros y Flyway se negará a arrancar.

11. Estrategia de pruebas

No pretendas cubrirlo todo. Con un equipo de tres y un semestre, apunta a esto:

Tipo	Herramienta	Qué probar exactamente
Unitaria de dominio	JUnit 5, sin Spring	Los métodos <code>negocio</code> de las entidades: <code>registrarAbono</code> , <code>seSolapaCon</code> , <code>puedeTransicionarA</code> , <code>registrarIntentoFallido</code> . Son rapidísimas y son las que más valor tienen.
Unitaria de servicio	JUnit 5 + Mockito	Que el servicio aplique las reglas y llame a lo que debe: por ejemplo, que <code>ReservaServiceImpl.solicitar</code> ejecute todos los validadores y publique el evento.
Integración de repositorio	<code>@DataJpaTest</code> + Testcontainers	Las consultas escritas a mano con <code>@Query</code> , sobre todo la de solapamiento.
Integración de API	<code>@SpringBootTest</code> + MockMvc	Un caso feliz y un caso de permiso denegado por cada módulo. Suficiente para demostrar seguridad.

Casos de prueba mínimos que deberías poder mostrar en la sustentación: pago que salda exactamente la obligación; pago

mayor al saldo rechazado; reserva de residente moroso rechazada; dos reservas simultáneas donde solo una queda aprobada; prorrateo con coeficientes que no suman 100 % rechazado; incidencia que no puede cerrarse sin solución registrada; quinto intento de login que bloquea la cuenta.

12. Hoja de ruta: nueve sprints

Cada sprint termina con algo **demostrable en Postman**. Si un sprint no produce un endpoint que funcione, está mal planteado. El orden no es caprichoso: cada módulo se apoya en lo construido antes.

Sprint	Objetivo	Qué construyes	Definición de hecho
0 1 sem.	Cimientos	Proyecto en Spring Initializr, docker-compose con PostgreSQL, perfiles dev/prod, EntidadBase, excepciones, GlobalExceptionHandler, Swagger, repositorio Git con ramas.	La aplicación arranca, conecta a la base y Swagger abre en /swagger-ui.html.
1 1-2 sem.	Inmueble tu rebanada de práctica	Torre y Apartamento completos: entidad, migración V1 parcial, repositorio, servicio, controlador, DTOs, mapper y tres pruebas.	Puedes crear, listar y consultar apartamentos por API. Aquí aprendes el patrón que repetirás seis veces.
2 2 sem.	Usuarios y seguridad RF01 · CU01, CU05	Jerarquía Usuario, SecurityConfig, JWT completo, login con bloqueo por intentos, registro de usuarios por el administrador, protección por rol.	Login devuelve token; un endpoint de administrador rechaza con 403 el token de un residente.
3 2 sem.	Cartera RF02 parte 1 · CU11	ObligacionFinanciera, generación de obligación mensual, estado de cuenta, marcado de vencidas con @Scheduled.	Un residente consulta su estado de cuenta y solo el suyo.
4 2 sem.	Pagos RF02 parte 2 · CU02, CU06	Jerarquía Pago, PasarelaSimulada primero y Wompi después, webhook, validación de pago manual, generación del comprobante PDF.	Pago aprobado deja saldo en cero y comprobante descargable. Es el sprint más pesado: no lo dejes para el final.
5 1-2 sem.	Reservas RF03, RF04 · CU03, CU07	ZonaComun, Reserva, los cuatro validadores, bloqueo optimista, aprobación y rechazo con justificación.	Una prueba con dos hilos simultáneos demuestra que solo una reserva sobrevive.
6 1-2 sem.	Mantenimiento y portería RF05, RF06 · CU04, CU08, CU09	Incidencias con máquina de estados, carga de evidencias con límite de tamaño, registro de ingresos y paquetería.	Ciclo completo de una incidencia: abierta → asignada → en proceso → resuelta.
7 1 sem.	Prorrrateo y comunicados RF07, RF08 · CU10, CU12	Cálculo por coeficientes con validación del 100 %, vista previa antes de confirmar, comunicados fijados.	El prorrrateo aparece desglosado en el estado de cuenta del residente.
8 1 sem.	Notificaciones y eventos	Eventos de dominio en los cuatro módulos, canales en plataforma y correo, campana de no leídas.	Aprobar una reserva genera notificación al residente sin que el módulo de reservas conozca comunicaciones.
9 1-2 sem.	Cierre y despliegue	Datos de demostración, revisión de permisos, documentación OpenAPI depurada, despliegue con Docker, preparación del guion de sustentación.	Un tercero levanta el backend con un comando y ejecuta el recorrido completo.

Cómo repartir con tu equipo. Tú, backend, deberías ir un sprint por delante del frontend. Entrega el contrato (rutas y forma del JSON) al inicio de cada sprint para que Cristian pueda maquetar contra datos simulados; así nadie queda bloqueado esperando al otro. En Jira, una historia por endpoint es demasiado fino: una historia por caso de uso funciona mejor.

13. Tu primera funcionalidad de punta a punta

Esta es la parte más importante del documento. Haz exactamente estos pasos con **Apartamento**, que es la entidad más simple del sistema. Cuando termines, habrás recorrido todas las capas y el resto será repetición con más reglas.

#	Paso	Detalle y criterio de que quedó bien
1	Levanta PostgreSQL con Docker	Un <code>docker-compose.yml</code> con usuario, contraseña y base <code>resigest</code> . Conéctate con DBeaver antes de seguir.
2	Genera el proyecto	Spring Initializr con las dependencias de la sección 1. Ejecuta <code>./mvnw spring-boot:run</code> : debe arrancar sin errores.
3	Configura <code>application-dev.yml</code>	URL de la base, <code>ddl-auto: validate</code> , <code>show-sql: true</code> . Ver el SQL que genera Hibernate es la mejor forma de aprender JPA.
4	Escribe <code>EntidadBase</code>	Los tres campos comunes y las anotaciones de auditoría. Es la primera clase real del proyecto.
5	Escribe la migración V1 con las tablas <code>torre</code> y <code>apartamento</code>	SQL a mano. Al arrancar, Flyway la aplica y crea la tabla <code>flyway_schema_history</code> .
6	Crea las entidades <code>Torre</code> y <code>Apartamento</code>	Con <code>@Entity</code> , la relación <code>@ManyToOne</code> y <code>fetch = LAZY</code> . Si arranca con <code>validate</code> , tu mapeo coincide con el SQL.
7	Crea <code>ApartamentoRepository</code>	Interfaz vacía que extiende <code>JpaRepository</code> . Sorpresa: ya tienes <code>save</code> , <code>findById</code> , <code>findAll</code> .
8	Crea los DTOs	<code>CrearApartamentoRequest</code> con validaciones y <code>ApartamentoDTO</code> de salida. Ambos como <code>record</code> .
9	Crea <code>ApartamentoService</code> y su implementación	Métodos <code>crear</code> , <code>obtener</code> , <code>listar</code> . En <code>obtener</code> , lanza <code>RecursoNoEncontradoException</code> si no existe.
10	Crea <code>ApartamentoController</code>	Tres endpoints. Ningún método debería pasar de cinco líneas: recibe, delega, devuelve.
11	Prueba en Postman	POST crea y devuelve 201; GET de un id inexistente devuelve 404 con el formato de <code>ApiError</code> .
12	Escribe tres pruebas	Una unitaria de <code>identificadorCompleto()</code> , una de servicio con Mockito, una de controlador con MockMvc.
13	Haz commit y abre un Pull Request	Rama <code>feature/modulo-inmuble</code> . Que un compañero lo revise: acostúmbrense desde ahora.

Terminado esto, el módulo de usuarios es el mismo procedimiento más herencia y seguridad; el financiero es el mismo procedimiento más reglas de negocio. **Ya no vas a estar perdido: vas a estar repitiendo un patrón que conoces.**

14. Errores frecuentes y cómo evitarlos

Error	Por qué duele y qué hacer
Devolver entidades JPA desde el controlador	Provoca recursión infinita en el JSON (<code>apartamento</code> → <code>residentes</code> → <code>apartamento</code>) y expone el hash de la contraseña. Solución: DTO siempre, sin excepciones.
Poner las reglas en el controlador	Quedan imposibles de probar y se duplican cuando aparece un segundo endpoint. El controlador solo traduce HTTP.
Usar <code>double</code> para dinero	<code>0.1 + 0.2</code> no da <code>0.3</code> . En cartera eso son reclamos de residentes. <code>BigDecimal</code> , con escala 2 y <code>RoundingMode.HALF_UP</code> .
Prorratar dividiendo y redondeando cada parte	Las partes no suman el total y queda un centavo bailando. Calcula todas las partes, súmalas y asigna la diferencia al apartamento de mayor coeficiente.
<code>ddl-auto: update</code> en cualquier ambiente	Hibernate agrega columnas pero nunca las quita ni corrige tipos; el esquema se desincroniza en silencio. Usa Flyway y <code>validate</code> .
Relaciones <code>EAGER</code> por defecto	Una consulta de apartamentos termina trayendo media base. Todo <code>LAZY</code> ; cuando necesites traer lo relacionado, usa <code>JOIN FETCH</code> explícito.
Guardar el secreto del JWT o las llaves de Wompi en el repositorio	Quedan en el historial de Git para siempre. Variables de entorno y un <code>.env</code> ignorado.
Dejar la seguridad para el final	Meter autenticación sobre 40 endpoints ya escritos obliga a reescribir servicios y pruebas. Va en el sprint 2, no en el 9.
Confiar en el retorno del navegador para dar un pago por aprobado	El usuario puede cerrar la pestaña y el pago igual se aprueba, o falsear la vuelta. La verdad es el webhook con firma validada.
Ramas gigantes de dos semanas	La integración se vuelve una pesadilla. Una rama por caso de uso, mezclada en pocos días.

Anexo A. Checklist de avance

Cimientos

- [] Proyecto generado y arrancando
- [] PostgreSQL en Docker y conexión verificada
- [] Perfiles dev y prod separados
- [] EntidadBase con auditoría
- [] Jerarquía de excepciones + GlobalExceptionHandler
- [] Flyway con V1 aplicada
- [] Swagger accesible
- [] Repositorio Git con ramas y .gitignore correcto

Por cada módulo (repite ocho veces)

- [] Entidades y enums creados
- [] Migración SQL escrita y aplicada
- [] Repositorio con las consultas necesarias
- [] Interfaz de servicio + implementación
- [] DTOs de entrada con validación y de salida
- [] Mapper
- [] Controlador con permisos por rol y por propietario
- [] Reglas de negocio en los métodos de la entidad
- [] Pruebas: dominio, servicio, endpoint
- [] Contrato compartido con el frontend
- [] Probado de punta a punta en Postman

Cierre

- [] Datos de demostración cargados (V6)
- [] Matriz de permisos verificada endpoint por endpoint
- [] Ningún secreto en el repositorio
- [] README con instrucciones de arranque
- [] Recorrido de demostración ensayado

Anexo B. Glosario técnico mínimo

Término	Qué significa en este proyecto
Bean	Objeto que Spring crea y administra por ti. Tus @Service, @Repository y @RestController son beans.
Inyección de dependencias	En vez de hacer <code>new</code> , declaras lo que necesitas en el constructor y Spring te lo entrega. Por eso los servicios se prueban fácil.
DTO	Objeto plano que viaja entre el backend y el frontend. No tiene lógica ni anotaciones de JPA.
Entidad	Clase mapeada a una tabla. Vive dentro del backend y nunca sale por el API.
Repositorio	Interfaz de acceso a datos que Spring Data implementa a partir del nombre de sus métodos.
@Transactional	Marca un método de servicio para que todo lo que haga se confirme junto o se revierta junto.
Bloqueo optimista	Técnica que detecta si otro usuario modificó la misma fila mientras trabajabas, usando una columna <code>version</code> .
Webhook	Petición que un servicio externo (Wompi) hace <i>hacia tu servidor</i> para avisar que algo ocurrió.
JWT	Token firmado que el frontend envía en cada petición para demostrar quién es, sin que el servidor guarde sesión.
Migración	Archivo SQL versionado que describe un cambio del esquema. Se aplica una sola vez y nunca se edita.
Evento de dominio	Aviso interno de que algo relevante ocurrió, que otros módulos pueden escuchar sin acoplarse.

Última recomendación. El riesgo real de este proyecto no es técnico, es de secuencia: llegar a la semana diez con muchas entidades escritas y ningún flujo funcionando. Mide tu avance en **casos de uso que responden en Postman**, no en clases creadas. Una funcionalidad terminada de punta a punta vale más, para la nota y para tu aprendizaje, que cinco módulos a medio hacer.